

在 Flutter 應用程式加入 WebView

來源：<https://codelabs.developers.google.com/codelabs/flutter-webview?hl=zh-tw>

步驟數：13

在本程式碼研究室中，您將瞭解如何在 Flutter 應用程式中加入 `webview_flutter` 外掛程式。

目錄

1. 簡介
2. 設定 Flutter 開發環境
3. 開始使用
4. 在 Flutter 應用程式中新增 WebView 小工具
5. 監聽網頁載入事件
6. 使用 WebViewController
7. 使用 NavigationDelegate 追蹤導覽
8. 在 AppBar 中新增選單按鈕
9. 評估 JavaScript
10. 使用 JavaScript 管道
11. 管理 Cookie
12. 在 WebView 中載入 Flutter 資產、檔案和 HTML 字串
13. 大功告成！

步驟 1 · 約 0 分鐘

1. 簡介

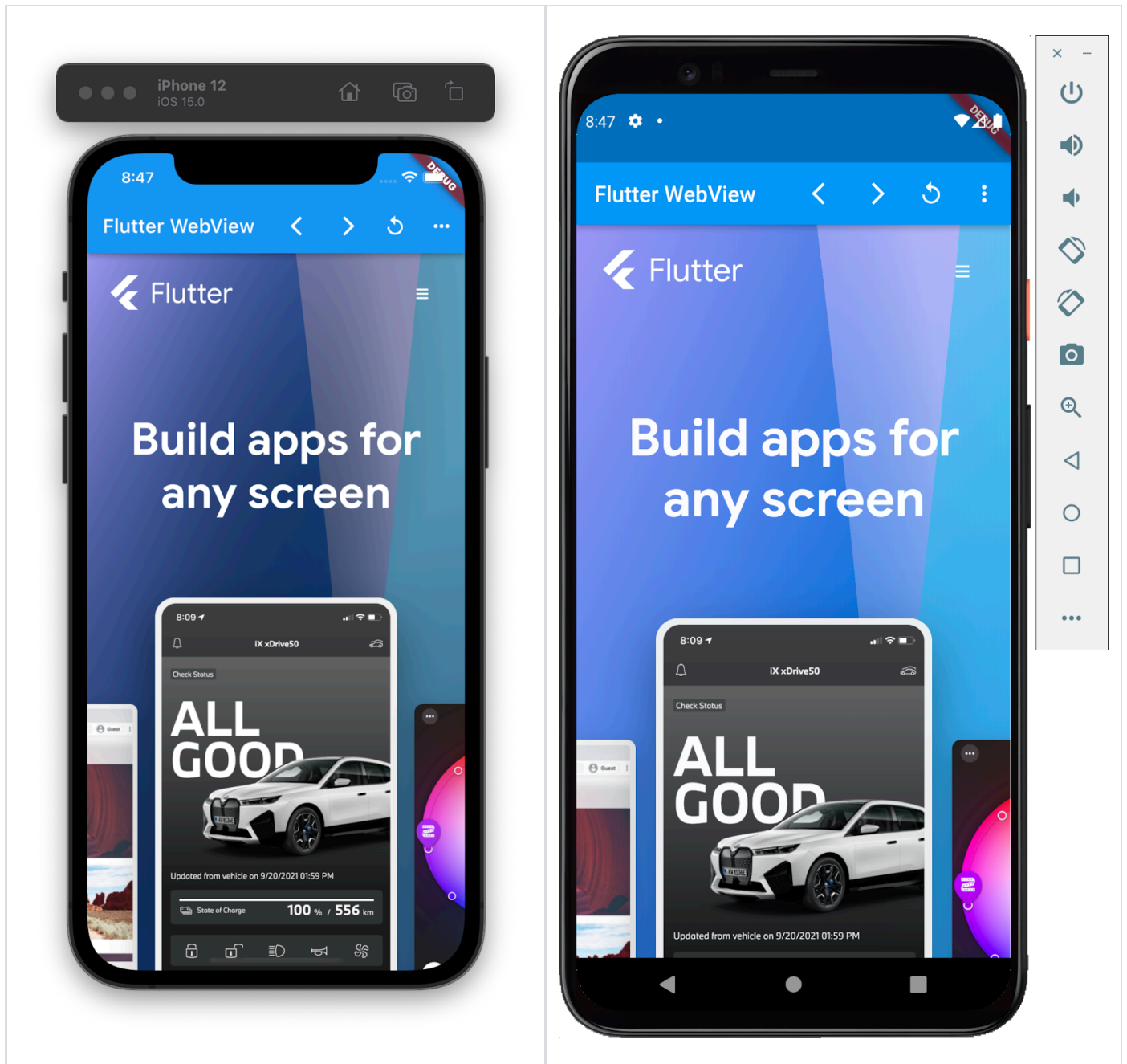
上次更新時間：2021 年 10 月 19 日

您可以使用 [WebView Flutter 外掛程式](#)，在 Android 或 iOS Flutter 應用程式中加入 WebView 小工具。在 iOS 上，WebView 小工具是由 [WKWebView](#) 支援，而在 Android 上，WebView 小工具是由 [WebView](#) 支援。外掛程式可以在網頁檢視畫面中算繪 Flutter 小工具。舉例來說，您可以在網頁檢視畫面中算繪下拉式選單。

建構項目

在本程式碼研究室中，您將使用 Flutter SDK 逐步建構行動應用程式，並加入 WebView。您的應用程式將會：

- 在 `WebView` 中顯示網頁內容
- 顯示堆疊在 `WebView` 上方的 Flutter 小工具
- 對網頁載入進度事件做出反應
- 透過 `WebViewController` 控制 `WebView`
- 使用 `NavigationDelegate` 封鎖網站
- 評估 JavaScript 運算式
- 使用 `JavascriptChannels` 處理 JavaScript 的回呼
- 設定、移除、新增或顯示 Cookie
- 從素材資源、檔案或包含 HTML 的字串載入及顯示 HTML



課程內容

在本程式碼研究室中，您將瞭解如何以各種方式使用 `webview_flutter` 外掛程式，包括：

- 如何設定 `webview_flutter` 外掛程式
- 如何監聽網頁載入進度事件
- 如何控制頁面導覽
- 如何命令 `WebView` 在歷史記錄中前後瀏覽
- 如何評估 JavaScript，包括使用傳回的結果
- 如何註冊回呼，從 JavaScript 呼叫 Dart 程式碼
- 如何管理 Cookie
- 如何從素材資源、檔案或包含 HTML 的字串載入及顯示 HTML 網頁

軟硬體需求

- Android Studio 4.1 以上版本 (適用於 Android 開發)
 - Xcode 12 以上版本 (適用於 iOS 開發)
 - [Flutter SDK](#)
 - 程式碼編輯器，例如 [Android Studio](#) 或 [Visual Studio Code](#)。
-

步驟 2 · 約 10 分鐘

2. 設定 Flutter 開發環境

如要完成本實驗室，您需要兩項軟體：[Flutter SDK](#) 和[編輯器](#)。

您可以使用下列任一裝置執行程式碼研究室：

- 連線至電腦並設為開發人員模式的實體 [Android](#) 或 [iOS](#) 裝置。
 - [iOS 模擬器](#) (需要安裝 Xcode 工具)。
 - [Android Emulator](#) (需在 Android Studio 中設定)。
-

步驟 3 · 約 10 分鐘

3. 開始使用

開始使用 Flutter

建立新的 Flutter 專案有多種方法，Android Studio 和 Visual Studio Code 都提供這項工作的工具。您可以按照連結的程序建立專案，也可以在方便的指令列終端機中執行下列指令。

```
$ flutter create --platforms=android,ios webview_in_flutter
Creating project webview_in_flutter...
Resolving dependencies in `webview_in_flutter`...
Downloading packages...
Got dependencies in `webview_in_flutter`.
Wrote 74 files.

All done!
You can find general documentation for Flutter at: https://docs.flutter.dev/
Detailed API documentation is available at: https://api.flutter.dev/
If you prefer video documentation, consider: https://www.youtube.com/c/flutterdev

In order to run your application, type:

$ cd webview_in_flutter
$ flutter run

Your application code is in webview_in_flutter/lib/main.dart.
```

將 WebView Flutter 外掛程式新增為依附元件

如要為 Flutter 應用程式新增其他功能，最好使用 [Pub 套件](#)。在本程式碼研究室中，您將在專案中新增 `webview_flutter` 外掛程式。在終端機中執行下列指令。

```
$ cd webview_in_flutter
$ flutter pub add webview_flutter
Resolving dependencies...
Downloading packages...
  collection 1.18.0 (1.19.0 available)
  leak_tracker 10.0.5 (10.0.7 available)
  leak_tracker_flutter_testing 3.0.5 (3.0.7 available)
  material_color_utilities 0.11.1 (0.12.0 available)
+ plugin_platform_interface 2.1.8
  string_scanner 1.2.0 (1.3.0 available)
  test_api 0.7.2 (0.7.3 available)
+ webview_flutter 4.9.0
+ webview_flutter_android 3.16.7
+ webview_flutter_platform_interface 2.10.0
+ webview_flutter_wkwebview 3.15.0
Changed 5 dependencies!
6 packages have newer versions incompatible with dependency constraints.
Try `flutter pub outdated` for more information.
```

檢查 [pubspec.yaml](#) 時，您會發現依附元件區段中有一行 `webview_flutter` 外掛程式。

設定 Android minSDK

如要在 Android 上使用 `webview_flutter` 外掛程式，請將 `minSdk` 設為 `20`。按照下列方式修改 `android/app/build.gradle` 檔案：

[android/app/build.gradle](#)

```
android {
    //...

    defaultConfig {
        applicationId = "com.example.webview_in_flutter"
        minSdk = 20 // Modify this line
        targetSdk = flutter.targetSdkVersion
        versionCode = flutter.versionCode.toInteger()
        versionName = flutter.versionName
    }
}
```

步驟 4 · 約 5 分鐘

4. 在 Flutter 應用程式中新增 WebView 小工具

在這個步驟中，您要在應用程式中新增 `WebView`。WebView 是代管的內建檢視畫面，應用程式開發人員可選擇在應用程式中代管這些內建檢視畫面。在 Android 上，您可以選擇虛擬螢幕 (Android 的預設選項) 和混合組合。不過，iOS 一律使用混合組合。

如要深入瞭解虛擬螢幕和混合組合之間的差異，請參閱[這份文件](#)，瞭解如何在 Flutter 應用程式中透過平台檢視區塊代管原生 Android 和 iOS 檢視區塊。

在畫面上放置 WebView

將 `lib/main.dart` 改成以下內容：

[lib/main.dart](#)

```

import 'package:flutter/material.dart';
import 'package:webview_flutter/webview_flutter.dart';

void main() {
  runApp(
    MaterialApp(
      theme: ThemeData(useMaterial3: true),
      home: const WebViewApp(),
    ),
  );
}

class WebViewApp extends StatefulWidget {
  const WebViewApp({super.key});

  @override
  State<WebViewApp> createState() => _WebViewAppState();
}

class _WebViewAppState extends State<WebViewApp> {
  late final WebViewController controller;

  @override
  void initState() {
    super.initState();
    controller = WebViewController()
      ..loadRequest(
        Uri.parse('https://flutter.dev'),
      );
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Flutter WebView'),
      ),
      body: WebViewWidget(
        controller: controller,
      ),
    );
  }
}

```

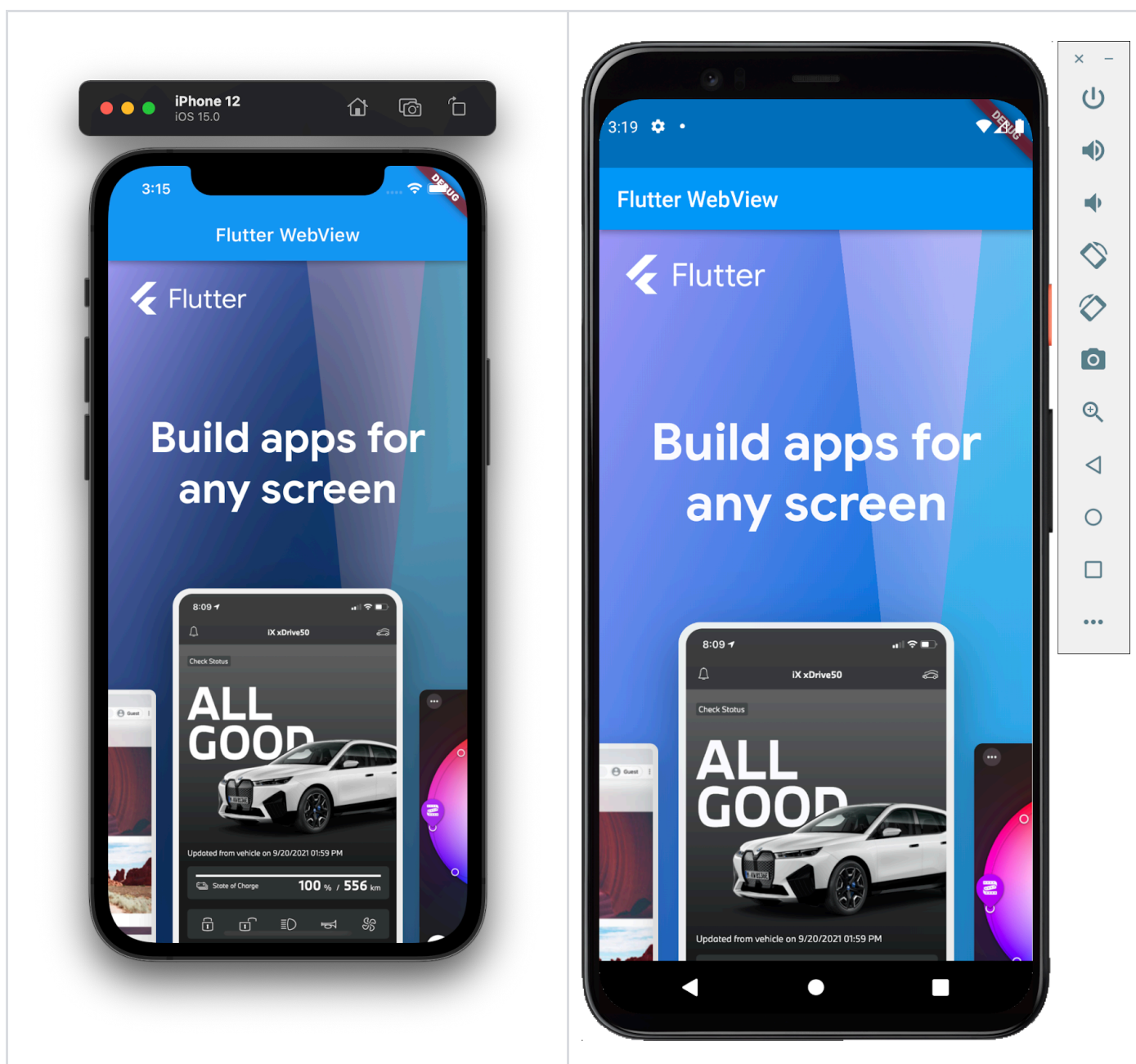
在 iOS 或 Android 上執行這項操作時，裝置會顯示 WebView，做為全出血瀏覽器視窗，也就是說，裝置會以全螢幕顯示瀏覽器，且沒有任何形式的邊框或邊界。捲動時，您會發現網頁的某些部分看起來有點奇怪。這是因為 JavaScript 已停用，而正確算繪 flutter.dev 需要 JavaScript。

執行應用程式

在 iOS 或 Android 中執行 Flutter 應用程式，即可看到顯示 flutter.dev 網站的 WebView。您也可以在 Android 模擬器或 iOS 模擬器中執行應用程式。您可以將初始 WebView 網址替換為自己的網站等。

```
$ flutter run
```

假設您已執行適當的模擬器或模擬器，或已附加實體裝置，將應用程式編譯並部署至裝置後，您應該會看到類似下列內容：



步驟 5 · 約 5 分鐘

5. 監聽網頁載入事件

`WebView` 小工具提供多個網頁載入進度事件，供應用程式監聽。在 `WebView` 網頁載入週期中，系統會觸發三種不同的網頁載入事件：`onPageStarted`、`onProgress` 和 `onPageFinished`。在這個步驟中，您將實作網頁載入指標。此外，這也會顯示您可以在 `WebView` 內容區域上算繪 Flutter 內容。

在應用程式中加入網頁載入事件

在 `lib/src/web_view_stack.dart` 建立新的來源檔案，並填入下列內容：

[lib/src/web_view_stack.dart](#)


```

import 'package:flutter/material.dart';
import 'package:webview_flutter/webview_flutter.dart';

class WebViewStack extends StatefulWidget {
  const WebViewStack({super.key});

  @override
  State<WebViewStack> createState() => _WebViewStackState();
}

class _WebViewStackState extends State<WebViewStack> {
  var loadingPercentage = 0;
  late final WebViewController controller;

  @override
  void initState() {
    super.initState();
    controller = WebViewController()
      ..setNavigationDelegate(NavigationDelegate(
        onPageStarted: (url) {
          setState(() {
            loadingPercentage = 0;
          });
        },
        onProgress: (progress) {
          setState(() {
            loadingPercentage = progress;
          });
        },
        onPageFinished: (url) {
          setState(() {
            loadingPercentage = 100;
          });
        },
      ))
      ..loadRequest(
        Uri.parse('https://flutter.dev'),
      );
  }

  @override
  Widget build(BuildContext context) {
    return Stack(
      children: [
        WebViewWidget(
          controller: controller,
        ),
        if (loadingPercentage < 100)
          LinearProgressIndicator(
            value: loadingPercentage / 100.0,
          ),
      ],
    );
  }
}

```

```
);  
}  
}
```

這段程式碼已將 `WebView` 小工具包裝在 `Stack` 中，當網頁載入百分比小於 100% 時，會以 `LinearProgressIndicator` 覆蓋 `WebView`。由於這涉及隨時間變更的程式狀態，因此您已將此狀態儲存在與 `StatefulWidget` 相關聯的 `State` 類別中。

如要使用這個新的 `WebViewStack` 小工具，請按照下列方式修改 `lib/main.dart`：

lib/main.dart

```
import 'package:flutter/material.dart';  
  
import 'src/web_view_stack.dart';  
  
void main() {  
  runApp(  
    MaterialApp(  
      theme: ThemeData(useMaterial3: true),  
      home: const WebViewApp(),  
    ),  
  );  
}  
  
class WebViewApp extends StatefulWidget {  
  const WebViewApp({super.key});  
  
  @override  
  State<WebViewApp> createState() => _WebViewAppState();  
}  
  
class _WebViewAppState extends State<WebViewApp> {  
  @override  
  Widget build(BuildContext context) {  
    return Scaffold(  
      appBar: AppBar(  
        title: const Text('Flutter WebView'),  
      ),  
      body: const WebViewStack(),  
    );  
  }  
}
```

執行應用程式時，視網路狀況而定，以及瀏覽器是否已快取您要前往的網頁，您會在 `WebView` 內容區域上方看到網頁載入指標。

步驟 6 · 約 5 分鐘

6. 使用 WebViewController

從 WebView 小工具存取 WebViewController

`WebView` 小工具可透過 `WebViewController` 進行程式輔助控制。建構 `WebView` 小工具後，系統會透過回呼提供這個控制器。由於這個控制器的可用性屬於非同步性質，因此非常適合做為 Dart 的非同步 `Completer<T>` 類別。

按照下列方式更新 `lib/src/web_view_stack.dart`：

[lib/src/web_view_stack.dart](#)

```

import 'package:flutter/material.dart';
import 'package:webview_flutter/webview_flutter.dart';

class WebViewStack extends StatefulWidget {
  const WebViewStack({required this.controller, super.key}); // MODIFY

  final WebViewController controller; // ADD

  @override
  State<WebViewStack> createState() => _WebViewStackState();
}

class _WebViewStackState extends State<WebViewStack> {
  var loadingPercentage = 0;
  // REMOVE the controller that was here

  @override
  void initState() {
    super.initState();
    // Modify from here...
    widget.controller.setNavigationDelegate(
      NavigationDelegate(
        onPageStarted: (url) {
          setState(() {
            loadingPercentage = 0;
          });
        },
        onProgress: (progress) {
          setState(() {
            loadingPercentage = progress;
          });
        },
        onPageFinished: (url) {
          setState(() {
            loadingPercentage = 100;
          });
        },
      ),
    );
    // ...to here.
  }

  @override
  Widget build(BuildContext context) {
    return Stack(
      children: [
        WebViewWidget(
          controller: widget.controller, // MODIFY
        ),
        if (loadingPercentage < 100)
          LinearProgressIndicator(
            value: loadingPercentage / 100.0,

```

```
        ),  
      ],  
    );  
  }  
}
```

`WebViewStack` 小工具現在會使用在周圍小工具中建立的控制器。這樣一來，`WebViewWidget` 的控制器就能與應用程式的其他部分共用。

製作導覽控制選項

擁有可運作的 `WebView` 是一回事，但能夠在網頁歷史記錄中前後瀏覽及重新載入網頁，則是一組實用的附加功能。幸好，您可以使用 `WebViewController` 將這項功能新增至應用程式。

在 `lib/src/navigation_controls.dart` 建立新的來源檔案，並填入下列內容：

[lib/src/navigation_controls.dart](#)

```

import 'package:flutter/material.dart';
import 'package:webview_flutter/webview_flutter.dart';

class NavigationControls extends StatelessWidget {
  const NavigationControls({required this.controller, super.key});

  final WebViewController controller;

  @override
  Widget build(BuildContext context) {
    return Row(
      children: <Widget>[
        IconButton(
          icon: const Icon(Icons.arrow_back_ios),
          onPressed: () async {
            final messenger = ScaffoldMessenger.of(context);
            if (await controller.canGoBack()) {
              await controller.goBack();
            } else {
              messenger.showSnackBar(
                const SnackBar(content: Text('No back history item')),
              );
              return;
            }
          },
        ),
        IconButton(
          icon: const Icon(Icons.arrow_forward_ios),
          onPressed: () async {
            final messenger = ScaffoldMessenger.of(context);
            if (await controller.canGoForward()) {
              await controller.goForward();
            } else {
              messenger.showSnackBar(
                const SnackBar(content: Text('No forward history item')),
              );
              return;
            }
          },
        ),
        IconButton(
          icon: const Icon(Icons.replay),
          onPressed: () {
            controller.reload();
          },
        ),
      ],
    );
  }
}

```

這個小工具會使用建構時與其共用的 `WebViewController`，讓使用者透過一系列 `IconButton` 控制 `WebView`。

在 AppBar 中新增導覽控制項

現在您已取得更新後的 `WebViewStack` 和新製作的 `NavigationControls`，接下來請將所有內容整合到更新後的 `WebViewApp` 中。我們將在此建構共用 `WebViewController`。由於 `WebViewApp` 位於這個應用程式的 Widget 樹狀結構頂端附近，因此在這個層級建立是合理的做法。

按照下列方式更新 `lib/main.dart` 檔案：

`lib/main.dart`

```

import 'package:flutter/material.dart';
import 'package:webview_flutter/webview_flutter.dart'; // ADD

import 'src/navigation_controls.dart'; // ADD
import 'src/web_view_stack.dart';

void main() {
  runApp(
    MaterialApp(
      theme: ThemeData(useMaterial3: true),
      home: const WebViewApp(),
    ),
  );
}

class WebViewApp extends StatefulWidget {
  const WebViewApp({super.key});

  @override
  State<WebViewApp> createState() => _WebViewAppState();
}

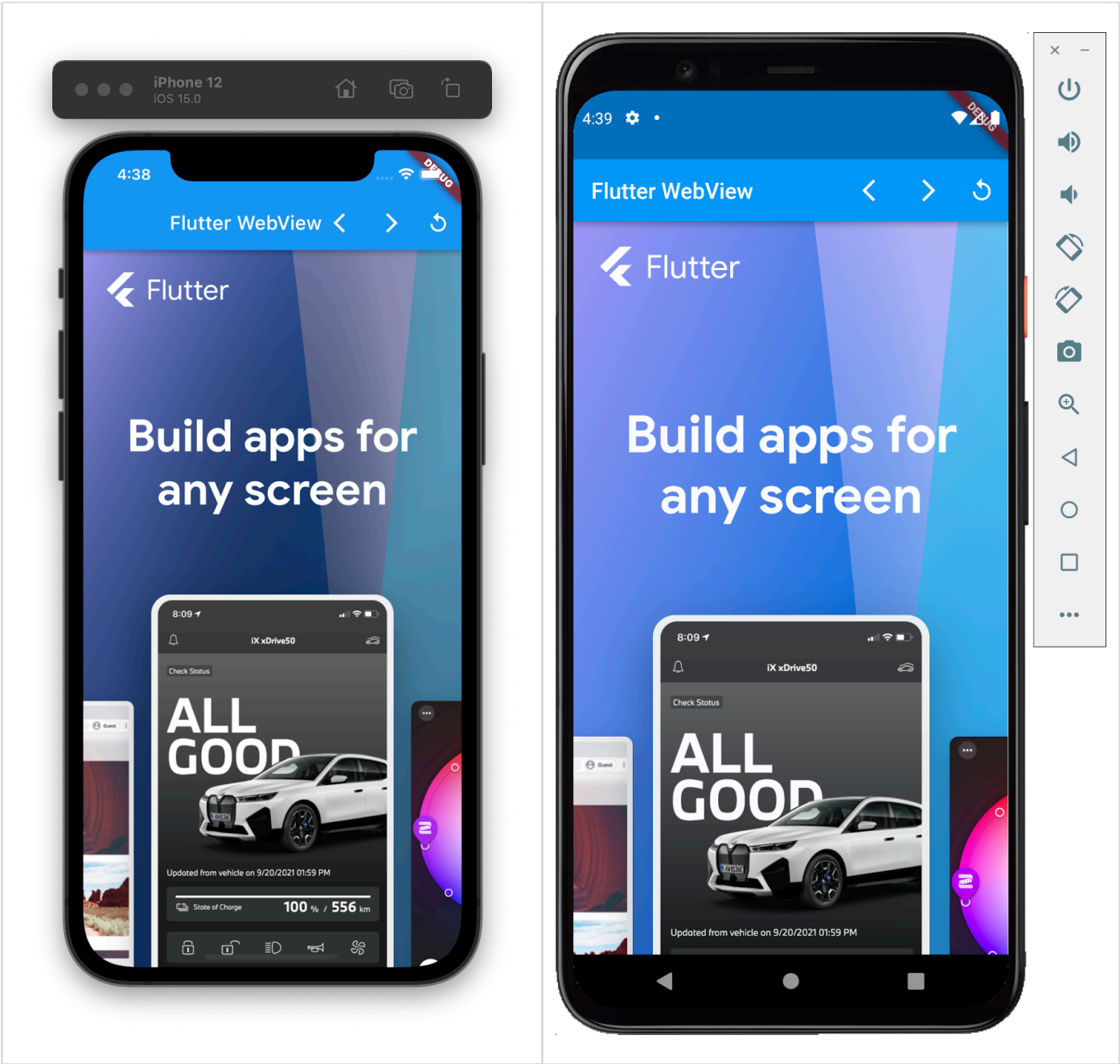
class _WebViewAppState extends State<WebViewApp> {
  // Add from here...
  late final WebViewController controller;

  @override
  void initState() {
    super.initState();
    controller = WebViewController()
      ..loadRequest(
        Uri.parse('https://flutter.dev'),
      );
  }
  // ...to here.

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Flutter WebView'),
        // Add from here...
        actions: [
          NavigationControls(controller: controller),
        ],
        // ...to here.
      ),
      body: WebViewStack(controller: controller), // MODIFY
    );
  }
}

```


執行應用程式後，應該會顯示含有控制項的網頁：



步驟 7 · 約 5 分鐘

7. 使用 `NavigationDelegate` 追蹤導覽

`WebView` 會為應用程式提供 `NavigationDelegate`，讓應用程式追蹤及控制 `WebView` 小工具的網頁瀏覽作業。當 `WebView` 啟動導覽時 (例如使用者點選連結)，系統會呼叫 `NavigationDelegate`。`NavigationDelegate` 回呼可用來控管 `WebView` 是否繼續導覽。

註冊自訂 `NavigationDelegate`

在這個步驟中，您將註冊 `NavigationDelegate` 回呼，以封鎖前往 [YouTube.com](https://www.youtube.com) 的導覽。請注意，這個簡化的實作方式也會封鎖內嵌的 YouTube 內容，這類內容會出現在各種 Flutter API 說明文件頁面中。

將 `lib/src/web_view_stack.dart` 更新為以下內容：

[lib/src/web_view_stack.dart](#)

```

import 'package:flutter/material.dart';
import 'package:webview_flutter/webview_flutter.dart';

class WebViewStack extends StatefulWidget {
  const WebViewStack({required this.controller, super.key});

  final WebViewController controller;

  @override
  State<WebViewStack> createState() => _WebViewStackState();
}

class _WebViewStackState extends State<WebViewStack> {
  var loadingPercentage = 0;

  @override
  void initState() {
    super.initState();
    widget.controller.setNavigationDelegate(
      NavigationDelegate(
        onPageStarted: (url) {
          setState(() {
            loadingPercentage = 0;
          });
        },
        onProgress: (progress) {
          setState(() {
            loadingPercentage = progress;
          });
        },
        onPageFinished: (url) {
          setState(() {
            loadingPercentage = 100;
          });
        },
        // Add from here...
        onNavigationRequest: (navigation) {
          final host = Uri.parse(navigation.url).host;
          if (host.contains('youtube.com')) {
            ScaffoldMessenger.of(context).showSnackBar(
              SnackBar(
                content: Text(
                  'Blocking navigation to $host',
                ),
              ),
            );
            return NavigationDecision.prevent;
          }
          return NavigationDecision.navigate;
        },
        // ...to here.
      ),
    );
  }
}

```

```
    );  
}  
  
@override  
Widget build(BuildContext context) {  
    return Stack(  
        children: [  
            WebViewWidget(  
                controller: widget.controller,  
            ),  
            if (loadingPercentage < 100)  
                LinearProgressIndicator(  
                    value: loadingPercentage / 100.0,  
                ),  
        ],  
    );  
}  
}
```

在下一個步驟中，您將新增選單項目，以便使用 `WebViewController` 類別測試 `NavigationDelegate`。讀者可自行練習擴增回呼的邏輯，只封鎖前往 YouTube.com 的全頁面導覽，但仍允許 API 說明文件中的內嵌 YouTube 內容。

8. 在 AppBar 中新增選單按鈕

在接下來的幾個步驟中，您將在 `AppBar` 小工具中製作選單按鈕，用於評估 JavaScript、叫用 JavaScript 管道及管理 Cookie。總而言之，這確實是實用的選單。

在 `lib/src/menu.dart` 建立新的來源檔案，並填入下列內容：

`lib/src/menu.dart`

```
import 'package:flutter/material.dart';
import 'package:webview_flutter/webview_flutter.dart';

enum _MenuOptions {
  navigationDelegate,
}

class Menu extends StatelessWidget {
  const Menu({required this.controller, super.key});

  final WebViewController controller;

  @override
  Widget build(BuildContext context) {
    return PopupMenuButton<_MenuOptions>(
      onSelect: (value) async {
        switch (value) {
          case _MenuOptions.navigationDelegate:
            await controller.loadRequest(Uri.parse('https://youtube.com'));
        }
      },
      itemBuilder: (context) => [
        const PopupMenuItem<_MenuOptions>(
          value: _MenuOptions.navigationDelegate,
          child: Text('Navigate to YouTube'),
        ),
      ],
    );
  }
}
```

使用者選取「前往 YouTube」選單選項時，系統會執行 `WebViewController` 的 `loadRequest` 方法。您在上一個步驟中建立的 `navigationDelegate` 回呼會封鎖這項導覽。

如要將選單新增至 `WebViewApp` 的畫面，請按照下列方式修改 `lib/main.dart`：

`lib/main.dart`

```

import 'package:flutter/material.dart';
import 'package:webview_flutter/webview_flutter.dart';

import 'src/menu.dart'; // ADD
import 'src/navigation_controls.dart';
import 'src/web_view_stack.dart';

void main() {
  runApp(
    MaterialApp(
      theme: ThemeData(useMaterial3: true),
      home: const WebViewApp(),
    ),
  );
}

class WebViewApp extends StatefulWidget {
  const WebViewApp({super.key});

  @override
  State<WebViewApp> createState() => _WebViewAppState();
}

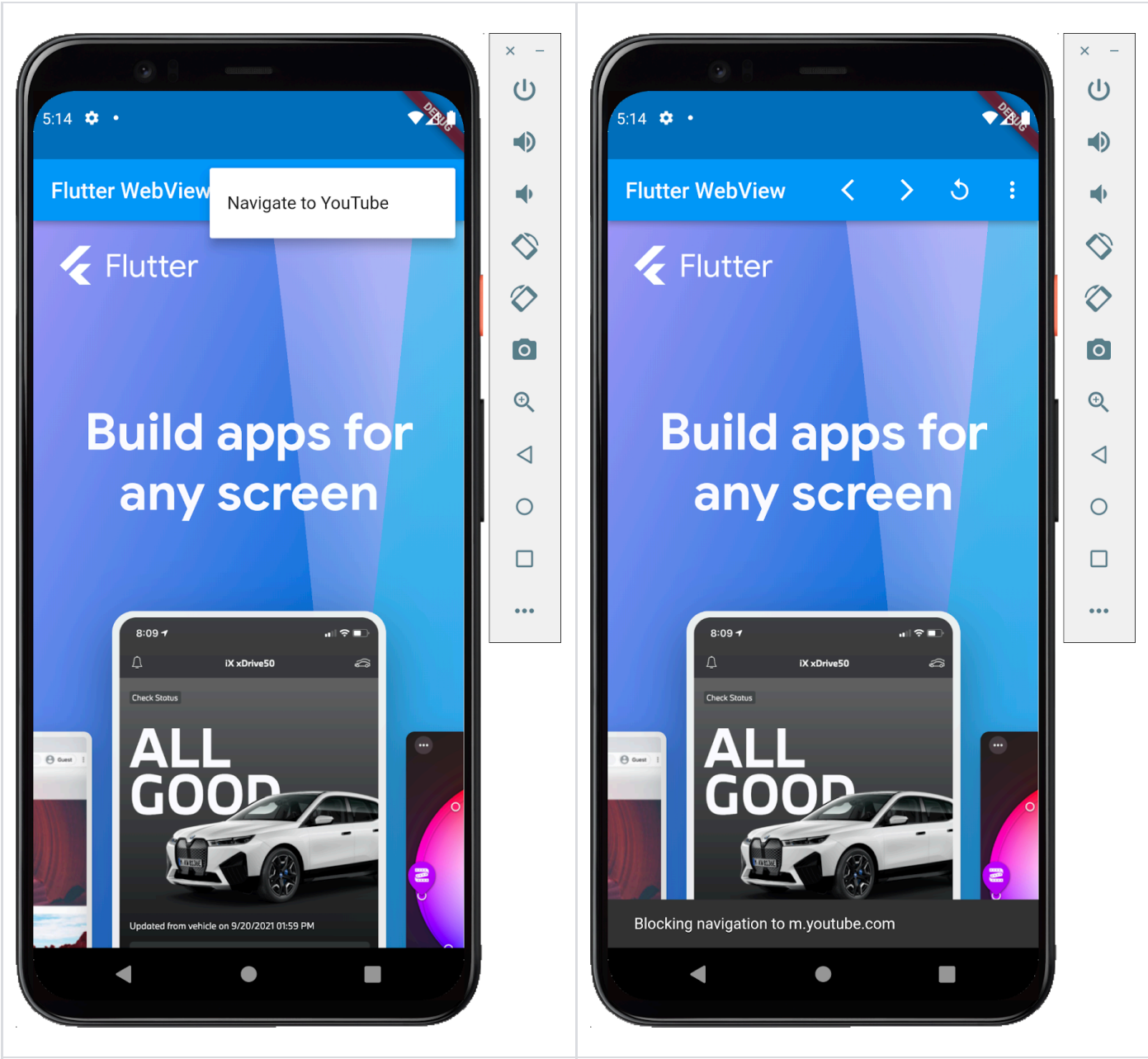
class _WebViewAppState extends State<WebViewApp> {
  late final WebViewController controller;

  @override
  void initState() {
    super.initState();
    controller = WebViewController()
      ..loadRequest(
        Uri.parse('https://flutter.dev'),
      );
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Flutter WebView'),
        actions: [
          NavigationControls(controller: controller),
          Menu(controller: controller), // ADD
        ],
      ),
      body: WebViewStack(controller: controller),
    );
  }
}

```

執行應用程式，然後輕觸「Navigate to YouTube」(前往 YouTube) 選單項目。系統應會顯示 SnackBar，通知您導覽控制器已封鎖前往 YouTube 的導覽作業。



步驟 9 · 約 5 分鐘

9. 評估 JavaScript

`WebViewController` 可評估目前網頁環境中的 JavaScript 運算式。評估 JavaScript 的方式有兩種：對於不會傳回值的 JavaScript 程式碼，請使用 `runJavaScript`；對於會傳回值的 JavaScript 程式碼，請使用 `runJavaScriptReturningResult`。

如要啟用 JavaScript，請將 `javascriptMode` 屬性設為 `JavascriptMode.unrestricted`，然後設定 `WebViewController`。根據預設，`javascriptMode` 會設為 `JavascriptMode.disabled`。

新增 `javascriptMode` 設定，更新 `_WebViewStackState` 類別，如下所示：

[lib/src/web_view_stack.dart](#)


```

class _WebViewStackState extends State<WebViewStack> {
  var loadingPercentage = 0;

  @override
  void initState() {
    super.initState();
    widget.controller
      ..setNavigationDelegate( // Modify this line to use .. instead of .
        NavigationDelegate(
          onPageStarted: (url) {
            setState(() {
              loadingPercentage = 0;
            });
          },
          onProgress: (progress) {
            setState(() {
              loadingPercentage = progress;
            });
          },
          onPageFinished: (url) {
            setState(() {
              loadingPercentage = 100;
            });
          },
          onNavigationRequest: (navigation) {
            final host = Uri.parse(navigation.url).host;
            if (host.contains('youtube.com')) {
              ScaffoldMessenger.of(context).showSnackBar(
                SnackBar(
                  content: Text(
                    'Blocking navigation to $host',
                  ),
                ),
              );
              return NavigationDecision.prevent;
            }
            return NavigationDecision.navigate;
          },
        ),
      )
      ..setJavaScriptMode(JavascriptMode.unrestricted); // Add this line
  }

  @override
  Widget build(BuildContext context) {
    return Stack(
      children: [
        WebViewWidget(
          controller: widget.controller,
        ),
        if (loadingPercentage < 100)
          LinearProgressIndicator(

```

```
        value: loadingPercentage / 100.0,  
      ),  
    ],  
  );  
}  
}
```

`WebViewWidget` 現在可以執行 JavaScript，因此您可以在選單中新增選項，使用 `runJavaScriptReturningResult` 方法。

使用編輯器或鍵盤，將 `Menu` 類別轉換為 `StatefulWidget`。修改 `lib/src/menu.dart`，使其符合下列內容：

[lib/src/menu.dart](#)

```

import 'package:flutter/material.dart';
import 'package:webview_flutter/webview_flutter.dart';

enum _MenuOptions {
  navigationDelegate,
  userAgent, // Add this line
}

class Menu extends StatefulWidget { // Convert to StatefulWidget
  const Menu({required this.controller, super.key});

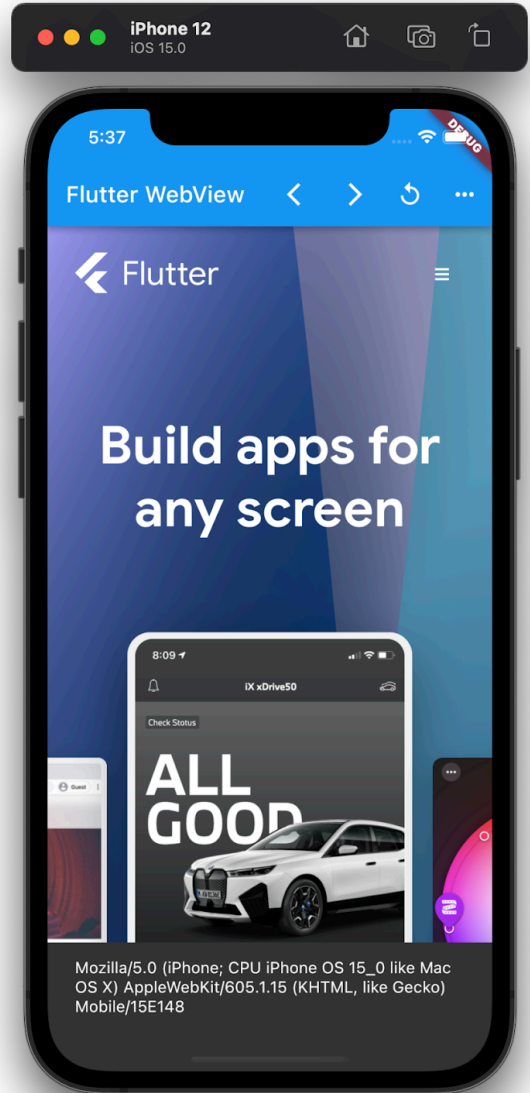
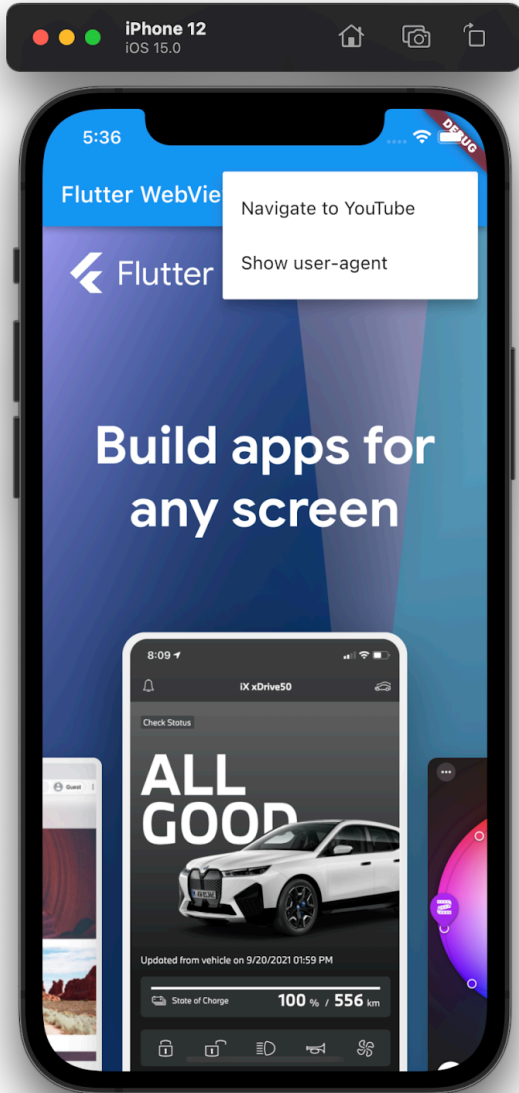
  final WebViewController controller;

  @override // Add from here
  State<Menu> createState() => _MenuState();
}

class _MenuState extends State<Menu> { // To here.
  @override
  Widget build(BuildContext context) {
    return PopupMenuButton<_MenuOptions>(
      onSelect: (value) async {
        switch (value) {
          case _MenuOptions.navigationDelegate: // Modify from here
            await widget.controller
              .loadRequest(Uri.parse('https://youtube.com'));
          case _MenuOptions.userAgent:
            final userAgent = await widget.controller
              .runJavaScriptReturningResult('navigator.userAgent');
            if (!context.mounted) return;
            ScaffoldMessenger.of(context).showSnackBar(SnackBar(
              content: Text('$userAgent'),
            )); // To here.
        }
      },
      itemBuilder: (context) => [
        const PopupMenuItem<_MenuOptions>(
          value: _MenuOptions.navigationDelegate,
          child: Text('Navigate to YouTube'),
        ),
        const PopupMenuItem<_MenuOptions>( // Add from here
          value: _MenuOptions.userAgent,
          child: Text('Show user-agent'),
        ), // To here.
      ],
    );
  }
}

```

輕觸「顯示 User-Agent」選單選項後，系統會在 `SnackBar` 中顯示執行 JavaScript 運算式 `navigator.userAgent` 的結果。執行應用程式時，您可能會發現 Flutter.dev 頁面看起來不太一樣。這是啟用 JavaScript 後的執行結果。



步驟 10 · 約 5 分鐘

10. 使用 JavaScript 管道

應用程式可透過 JavaScript 管道在 `WebViewWidget` 的 JavaScript 環境中註冊回呼處理常式，以便叫用這些常式，將值傳回應用程式的 Dart 程式碼。在這個步驟中，您將註冊 `SnackBar` 管道，該管道會透過 `XMLHttpRequest` 的結果呼叫。

將 `WebViewStack` 類別更新為以下內容：

[lib/src/web_view_stack.dart](#)

```

class WebViewStack extends StatefulWidget {
  const WebViewStack({required this.controller, super.key});

  final WebViewController controller;

  @override
  State<WebViewStack> createState() => _WebViewStackState();
}

class _WebViewStackState extends State<WebViewStack> {
  var loadingPercentage = 0;

  @override
  void initState() {
    super.initState();
    widget.controller
      ..setNavigationDelegate(
        NavigationDelegate(
          onPageStarted: (url) {
            setState(() {
              loadingPercentage = 0;
            });
          },
          onProgress: (progress) {
            setState(() {
              loadingPercentage = progress;
            });
          },
          onPageFinished: (url) {
            setState(() {
              loadingPercentage = 100;
            });
          },
          onNavigationRequest: (navigation) {
            final host = Uri.parse(navigation.url).host;
            if (host.contains('youtube.com')) {
              ScaffoldMessenger.of(context).showSnackBar(
                SnackBar(
                  content: Text(
                    'Blocking navigation to $host',
                  ),
                ),
              );
              return NavigationDecision.prevent;
            }
            return NavigationDecision.navigate;
          },
        ),
      )
    // Modify from here...
    ..setJavaScriptMode(JavaScriptMode.unrestricted)
    ..addJavaScriptChannel(

```

```

        'SnackBar',
        onMessageReceived: (message) {
            ScaffoldMessenger.of(context)
                .showSnackBar(SnackBar(content: Text(message.message)));
        },
    );
    // ...to here.
}

@override
Widget build(BuildContext context) {
    return Stack(
        children: [
            WebViewWidget(
                controller: widget.controller,
            ),
            if (loadingPercentage < 100)
                LinearProgressIndicator(
                    value: loadingPercentage / 100.0,
                ),
        ],
    );
}
}

```

在 `set` 中，每個 JavaScript 管道都會在 JavaScript 環境中提供管道物件，並以與 JavaScript 管道 `name` 相同的名稱做為視窗屬性。在 JavaScript 環境中使用此方法時，必須在 JavaScript 管道上呼叫 `postMessage`，傳送訊息給具名的 `JavascriptChannel` 的 `onMessageReceived` 回呼處理常式。

如要使用先前新增的 JavaScript 管道，請新增另一個選單項目，在 JavaScript 環境中執行 `XMLHttpRequest`，並使用 `SnackBar` JavaScript 管道傳回結果。

現在 `WebViewWidget` 知道我們的 JavaScript 管道，您將新增範例來進一步擴充應用程式。如要這麼做，請在 `Menu` 類別中新增額外的 `PopupMenuItem`，並新增額外功能。

新增 `javascriptChannel` 列舉值，並在 `Menu` 類別中新增實作項目，藉此更新 `_MenuOptions`，加入額外的選單選項，如下所示：

[lib/src/menu.dart](#)

```

enum _MenuOptions {
  navigationDelegate,
  userAgent,
  javascriptChannel, // Add this option
}

class Menu extends StatefulWidget {
  const Menu({required this.controller, super.key});

  final WebViewController controller;

  @override
  State<Menu> createState() => _MenuState();
}

class _MenuState extends State<Menu> {
  @override
  Widget build(BuildContext context) {
    return PopupMenuButton<_MenuOptions>(
      onSelect: (value) async {
        switch (value) {
          case _MenuOptions.navigationDelegate:
            await widget.controller
              .loadRequest(Uri.parse('https://youtube.com'));
          case _MenuOptions.userAgent:
            final userAgent = await widget.controller
              .runJavaScriptReturningResult('navigator.userAgent');
            if (!context.mounted) return;
            ScaffoldMessenger.of(context).showSnackBar(SnackBar(
              content: Text('$userAgent'),
            ));
          case _MenuOptions.javascriptChannel: // Add from here
            await widget.controller.runJavaScript('''
var req = new XMLHttpRequest();
req.open('GET', "https://api.ipify.org/?format=json");
req.onload = function() {
  if (req.status == 200) {
    let response = JSON.parse(req.responseText);
    SnackBar.postMessage("IP Address: " + response.ip);
  } else {
    SnackBar.postMessage("Error: " + req.status);
  }
}
req.send();'''); // To here.
        },
      itemBuilder: (context) => [
        const PopupMenuItem<_MenuOptions>(
          value: _MenuOptions.navigationDelegate,
          child: Text('Navigate to YouTube'),
        ),
        const PopupMenuItem<_MenuOptions>(

```



```

        value: _MenuOptions.userAgent,
        child: Text('Show user-agent'),
    ),
    const PopupMenuItem<_MenuOptions>( // Add from here
        value: _MenuOptions.javascriptChannel,
        child: Text('Lookup IP Address'),
    ), // To here.
  ],
);
}
}

```

使用者選擇「JavaScript Channel Example」(JavaScript 頻道範例) 選單選項時，系統會執行這段 JavaScript。

```

var req = new XMLHttpRequest();
req.open('GET', "https://api.ipify.org/?format=json");
req.onload = function() {
    if (req.status == 200) {
        SnackBar.postMessage(req.responseText);
    } else {
        SnackBar.postMessage("Error: " + req.status);
    }
}
req.send();

```

這段程式碼會將 GET 要求傳送至 Public IP Address API，並傳回裝置的 IP 位址。這個結果會顯示在 SnackBar 中，方法是在 SnackBar JavascriptChannel 上叫用 postMessage。

11. 管理 Cookie

應用程式可以使用 `CookieManager` 類別，在 `WebView` 中管理 Cookie。在這個步驟中，您將顯示 Cookie 清單、清除 Cookie 清單、刪除 Cookie，以及設定新的 Cookie。針對每個 Cookie 用途，在 `_MenuOptions` 中新增項目，如下所示：

lib/src/menu.dart

```
enum _MenuOptions {  
  navigationDelegate,  
  userAgent,  
  javascriptChannel,  
  // Add from here ...  
  listCookies,  
  clearCookies,  
  addCookie,  
  setCookie,  
  removeCookie,  
  // ... to here.  
}
```

這個步驟的其餘變更重點在於 `Menu` 類別，包括將 `Menu` 類別從無狀態轉換為有狀態。這項變更非常重要，因為 `Menu` 需要擁有 `CookieManager`，而無狀態小工具中的可變動狀態是不良的組合。

將 `CookieManager` 新增至產生的 `State` 類別，如下所示：

lib/src/menu.dart

```
class Menu extends StatefulWidget {  
  const Menu({required this.controller, super.key});  
  
  final WebViewController controller;  
  
  @override  
  State<Menu> createState() => _MenuState();  
}  
  
class _MenuState extends State<Menu> {  
  final cookieManager = WebViewCookieManager(); // Add this line  
  
  @override  
  Widget build(BuildContext context) {  
    // ...  
  }  
}
```

`_MenuState` 類別會包含先前在 `Menu` 類別中新增的程式碼，以及新加入的 `CookieManager`。在接下來的一系列章節中，您將在 `_MenuState` 中新增輔助函式，這些函式隨後會由尚未新增的選單項目叫用。

取得所有 Cookie 的清單

您將使用 JavaScript 取得所有 Cookie 的清單。如要達成這個目標，請在 `_MenuState` 類別結尾新增名為 `_onListCookies` 的輔助方法。使用 `runJavaScriptReturningResult` 方法，您的輔助方法會在 JavaScript 環境中執行 `document.cookie`，並傳回所有 Cookie 的清單。

將下列內容新增至 `_MenuState` 類別：

lib/src/menu.dart

```
Future<void> _onListCookies(WebViewController controller) async {
  final String cookies = await controller
    .runJavaScriptReturningResult('document.cookie') as String;
  if (!mounted) return;
  ScaffoldMessenger.of(context).showSnackBar(
    SnackBar(
      content: Text(cookies.isNotEmpty ? cookies : 'There are no cookies.'),
    ),
  );
}
```

清除所有 Cookie

如要清除 WebView 中的所有 Cookie，請使用 `CookieManager` 類別的 `clearCookies` 方法。如果 `CookieManager` 清除了 Cookie，這個方法會傳回解析為 `true` 的 `Future<bool>`；如果沒有可清除的 Cookie，則會傳回 `false`。

將下列內容新增至 `_MenuState` 類別：

lib/src/menu.dart

```
Future<void> _onClearCookies() async {
  final hadCookies = await cookieManager.clearCookies();
  String message = 'There were cookies. Now, they are gone!';
  if (!hadCookies) {
    message = 'There were no cookies to clear.';
  }
  if (!mounted) return;
  ScaffoldMessenger.of(context).showSnackBar(
    SnackBar(
      content: Text(message),
    ),
  );
}
```

新增 Cookie

如要新增 Cookie，請叫用 JavaScript。如要瞭解如何將 Cookie 新增至 JavaScript 文件，請參閱 [MDN 的深入說明](#)。

將下列內容新增至 `_MenuState` 類別：

lib/src/menu.dart

```
Future<void> _onAddCookie(WebViewController controller) async {
  await controller.runJavaScript('''var date = new Date();
  date.setTime(date.getTime()+(30*24*60*60*1000));
  document.cookie = "FirstName=John; expires=" + date.toGMTString();''');
  if (!mounted) return;
  ScaffoldMessenger.of(context).showSnackBar(
    const SnackBar(
      content: Text('Custom cookie added.'),
    ),
  );
}
```

使用 CookieManager 設定 Cookie

您也可以使用 [CookieManager](#) 設定 Cookie，如下所示。

將下列內容新增至 `_MenuState` 類別：

lib/src/menu.dart

```
Future<void> _onSetCookie(WebViewController controller) async {
  await cookieManager.setCookie(
    const WebViewCookie(name: 'foo', value: 'bar', domain: 'flutter.dev'),
  );
  if (!mounted) return;
  ScaffoldMessenger.of(context).showSnackBar(
    const SnackBar(
      content: Text('Custom cookie is set.'),
    ),
  );
}
```

移除 Cookie

如要移除 Cookie，請新增 Cookie，並將到期日設為過去日期。

將下列內容新增至 `_MenuState` 類別：

lib/src/menu.dart

```
Future<void> _onRemoveCookie(WebViewController controller) async {
  await controller.runJavaScript(
    'document.cookie="FirstName=John; expires=Thu, 01 Jan 1970 00:00:00 UTC" ';
  );
  if (!mounted) return;
  ScaffoldMessenger.of(context).showSnackBar(
    const SnackBar(
      content: Text('Custom cookie removed.'),
    ),
  );
}
```

新增 CookieManager 選單項目

剩下的工作是新增選單選項，並將這些選項連結至您剛才新增的輔助方法。將 `_MenuState` 類別更新為以下內容：

[lib/src/menu.dart](#)

```

class _MenuState extends State<Menu> {
  final cookieManager = WebViewCookieManager();

  @override
  Widget build(BuildContext context) {
    return PopupMenuButton<_MenuOptions>(
      onSelect: (value) async {
        switch (value) {
          case _MenuOptions.navigationDelegate:
            await widget.controller
              .loadRequest(Uri.parse('https://youtube.com'));
          case _MenuOptions.userAgent:
            final userAgent = await widget.controller
              .runJavaScriptReturningResult('navigator.userAgent');
            if (!context.mounted) return;
            ScaffoldMessenger.of(context).showSnackBar(SnackBar(
              content: Text('$userAgent'),
            ));
          case _MenuOptions.javascriptChannel:
            await widget.controller.runJavaScript(''
var req = new XMLHttpRequest();
req.open('GET', "https://api.ipify.org/?format=json");
req.onload = function() {
  if (req.status == 200) {
    let response = JSON.parse(req.responseText);
    SnackBar.postMessage("IP Address: " + response.ip);
  } else {
    SnackBar.postMessage("Error: " + req.status);
  }
}
req.send();''');

            case _MenuOptions.clearCookies: // Add from here
              await _onClearCookies();
            case _MenuOptions.listCookies:
              await _onListCookies(widget.controller);
            case _MenuOptions.addCookie:
              await _onAddCookie(widget.controller);
            case _MenuOptions.setCookie:
              await _onSetCookie(widget.controller);
            case _MenuOptions.removeCookie: // To here.
              await _onRemoveCookie(widget.controller);
          }
        },
      itemBuilder: (context) => [
        const PopupMenuItem<_MenuOptions>(
          value: _MenuOptions.navigationDelegate,
          child: Text('Navigate to YouTube'),
        ),
        const PopupMenuItem<_MenuOptions>(
          value: _MenuOptions.userAgent,
          child: Text('Show user-agent'),
        ),
      ],
    ),
  ),

```

```

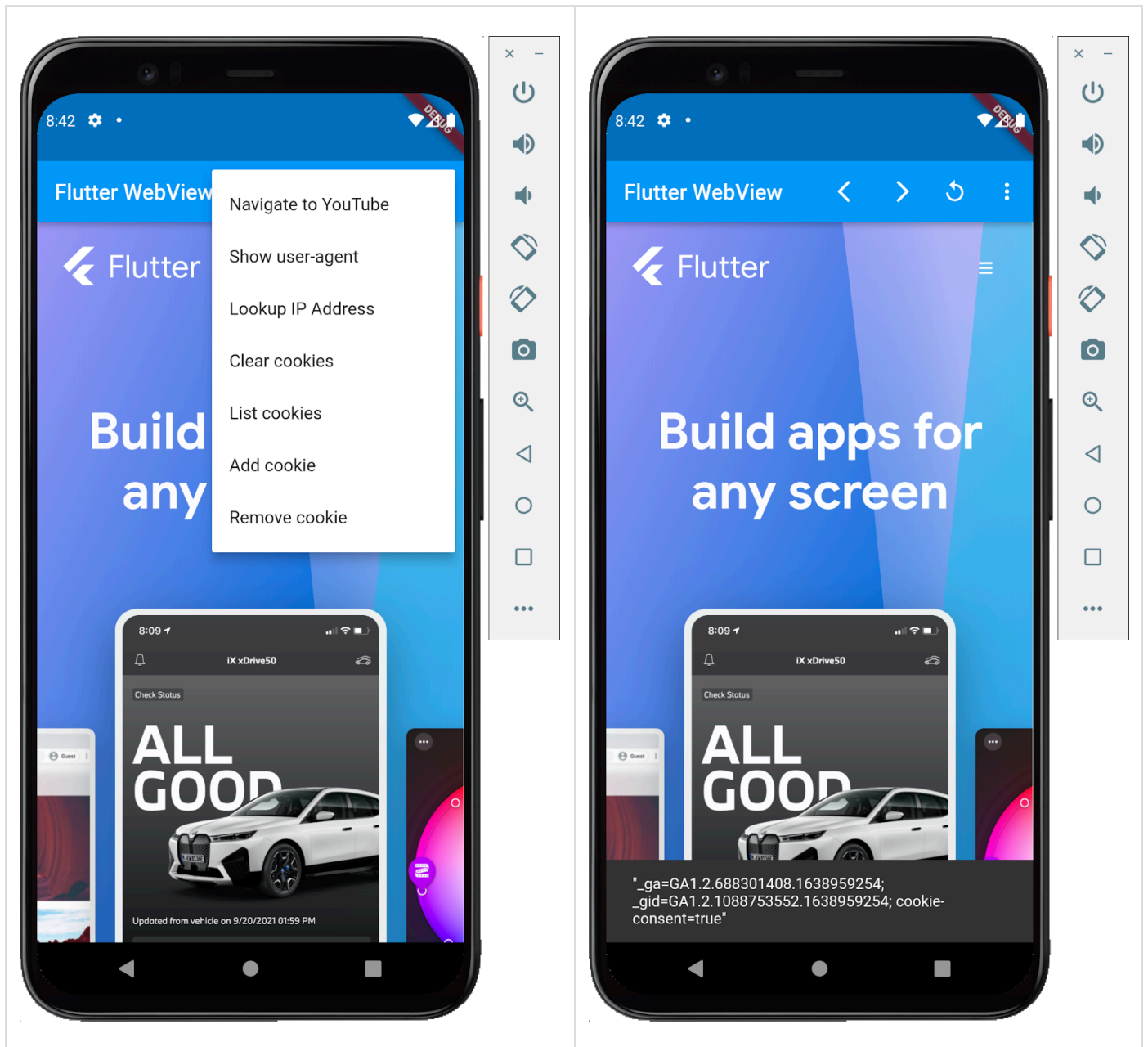
const PopupMenuItem<_MenuOptions>(
  value: _MenuOptions.javascriptChannel,
  child: Text('Lookup IP Address'),
),
const PopupMenuItem<_MenuOptions>(                                // Add from here
  value: _MenuOptions.clearCookies,
  child: Text('Clear cookies'),
),
const PopupMenuItem<_MenuOptions>(
  value: _MenuOptions.listCookies,
  child: Text('List cookies'),
),
const PopupMenuItem<_MenuOptions>(
  value: _MenuOptions.addCookie,
  child: Text('Add cookie'),
),
const PopupMenuItem<_MenuOptions>(
  value: _MenuOptions.setCookie,
  child: Text('Set cookie'),
),
const PopupMenuItem<_MenuOptions>(
  value: _MenuOptions.removeCookie,
  child: Text('Remove cookie'),
),                                // To here.
],
);
}

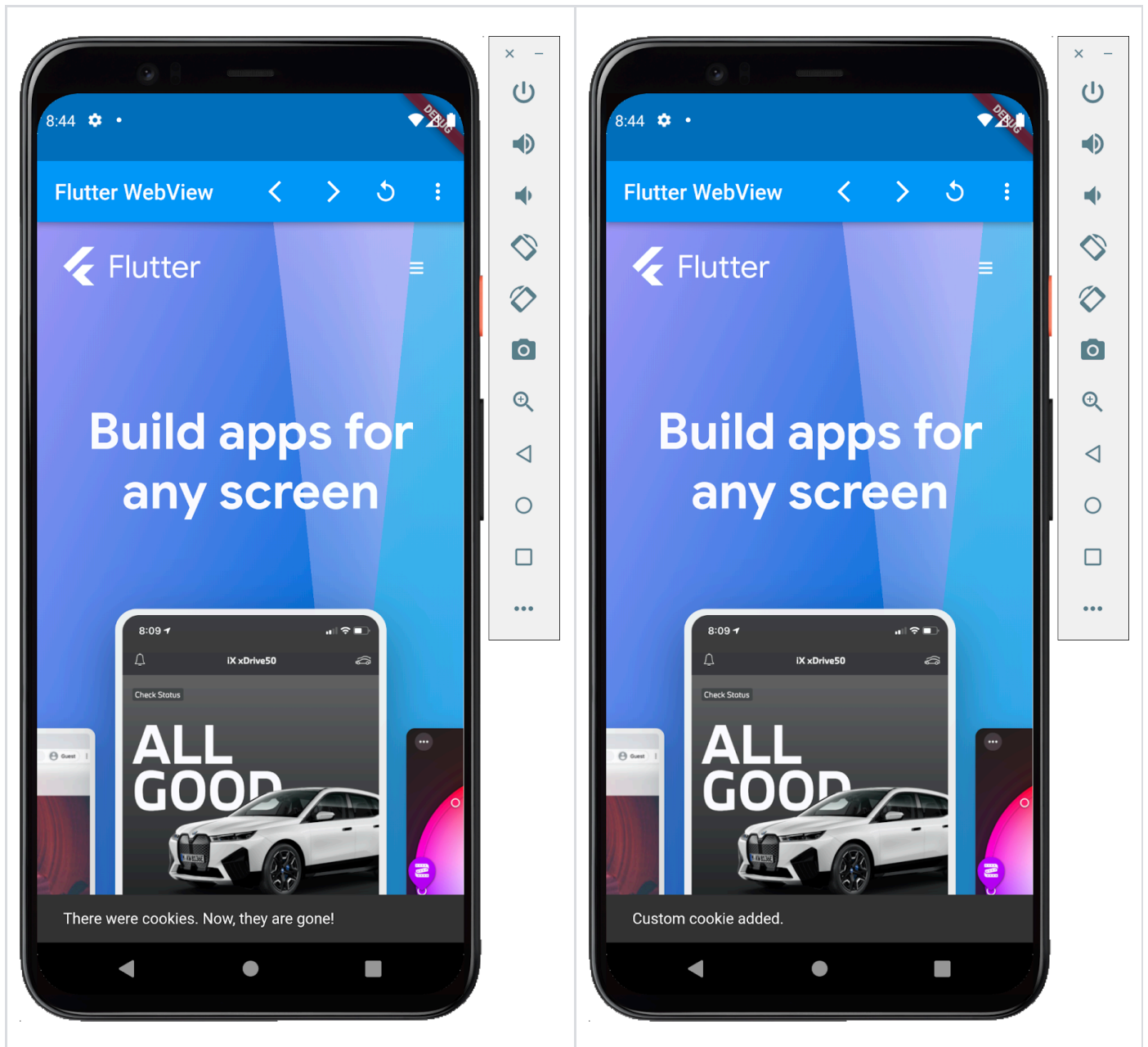
```

練習使用 CookieManager

如要使用您剛才新增至應用程式的所有功能，請嘗試下列步驟：

1. 選取「列出 Cookie」。其中應列出 flutter.dev 設定的 Google Analytics Cookie。
2. 選取「清除 Cookie」。這時應該會回報 Cookie 已清除。
3. 再次選取「清除 Cookie」。並回報沒有可清除的 Cookie。
4. 選取「列出 Cookie」。系統應會回報沒有任何 Cookie。
5. 選取「新增 Cookie」。並回報已新增 Cookie。
6. 選取「設定 Cookie」。系統應會回報已設定 Cookie。
7. 選取「列出 Cookie」，然後選取「移除 Cookie」。





12. 在 WebView 中載入 Flutter 資產、檔案和 HTML 字串

應用程式可使用不同方法載入 HTML 檔案，並在 WebView 中顯示。在這個步驟中，您將載入 `pubspec.yaml` 檔案中指定的 Flutter 資產、載入指定路徑中的檔案，以及使用 HTML 字串載入網頁。

如要載入位於指定路徑的檔案，請將 `path_provider` 新增至 `pubspec.yaml`。這是 Flutter 外掛程式，可尋找檔案系統中常用的位置。

在指令列中執行下列指令：

```
$ flutter pub add path_provider
```

如要載入資產，我們需要在 `pubspec.yaml` 中指定資產路徑。在 `pubspec.yaml` 中新增下列程式碼：

pubspec.yaml

```
# The following section is specific to Flutter packages.
flutter:

  # The following line ensures that the Material Icons font is
  # included with your application, so that you can use the icons in
  # the material Icons class.
  uses-material-design: true
  # Add from here
  assets:
    - assets/www/index.html
    - assets/www/styles/style.css
  # ... to here.
```

如要將素材資源新增至專案，請按照下列步驟操作：

1. 在專案的根資料夾中，建立名為 `assets` 的新目錄。
2. 在 `assets` 資料夾中，建立名為 `www` 的新目錄。
3. 在 `www` 資料夾中，建立名為 `styles` 的新目錄。
4. 在 `www` 資料夾中，建立名為 `index.html` 的新檔案。
5. 在 `styles` 資料夾中，建立名為 `style.css` 的新檔案。

複製下列程式碼並貼到 `index.html` 檔案中：

assets/www/index.html

```

<!DOCTYPE html>
<!-- Copyright 2013 The Flutter Authors. All rights reserved.
Use of this source code is governed by a BSD-style license that can be
found in the LICENSE file. -->
<html lang="en">
<head>
  <title>Load file or HTML string example</title>
  <link rel="stylesheet" href="styles/style.css" />
</head>
<body>

<h1>Local demo page</h1>
<p>
  This is an example page used to demonstrate how to load a local file or HTML
  string using the <a href="https://pub.dev/packages/webview_flutter">Flutter
  webview</a> plugin.
</p>

</body>
</html>

```

在 `style.css` 中，使用下列幾行程式碼設定 HTML 標頭樣式：

[assets/www/styles/style.css](#)

```

h1 {
  color: blue;
}

```

素材資源設定完成後，您就可以實作載入及顯示 Flutter 資產、檔案或 HTML 字串所需的方法。

載入 Flutter 資產

如要載入剛建立的資產，您只需要使用 `WebViewController` 呼叫 `loadFlutterAsset` 方法，並將資產路徑做為參數提供即可。在程式碼結尾新增下列方法：

[lib/src/menu.dart](#)

```

Future<void> _onLoadFlutterAssetExample(
  WebViewController controller, BuildContext context) async {
  await controller.loadFlutterAsset('assets/www/index.html');
}

```

載入本機檔案

如要在裝置上載入檔案，可以新增使用 `loadFile` 方法的方法，同樣是使用 `WebViewController`，其中會採用包含檔案路徑的 `String`。

您必須先建立包含 HTML 程式碼的檔案。方法是在 `menu.dart` 檔案的程式碼頂端，匯入項目正下方，將 HTML 程式碼新增為字串。

lib/src/menu.dart

```
import 'dart:io'; // Add this line,
import 'package:flutter/material.dart';
import 'package:path_provider/path_provider.dart'; // And this one.
import 'package:webview_flutter/webview_flutter.dart';

// Add from here ...
const String kExamplePage = '''
<!DOCTYPE html>
<html lang="en">
<head>
<title>Load file or HTML string example</title>
</head>
<body>

<h1>Local demo page</h1>
<p>
  This is an example page used to demonstrate how to load a local file or HTML
  string using the <a href="https://pub.dev/packages/webview_flutter">Flutter
  webview</a> plugin.
</p>

</body>
</html>
''';
// ... to here.
```

如要建立 `File` 並將 HTML 字串寫入檔案，請新增兩個方法。`_onLoadLocalFileExample` 會載入檔案，方法是提供 `_prepareLocalFile()` 方法傳回的路徑 (以字串形式)。在程式碼中新增下列方法：

lib/src/menu.dart

```
Future<void> _onLoadLocalFileExample(
  WebViewController controller, BuildContext context) async {
  final String pathToIndex = await _prepareLocalFile();

  await controller.loadFile(pathToIndex);
}

static Future<String> _prepareLocalFile() async {
  final String tmpDir = (await getTemporaryDirectory()).path;
  final File indexFile = File('$tmpDir/www/index.html');

  await Directory('$tmpDir/www').create(recursive: true);
  await indexFile.writeAsString(kExamplePage);

  return indexFile.path;
}
```

載入 HTML 字串

如要透過提供 HTML 字串來顯示網頁，程序相當簡單。`WebViewController` 有一個名為 `loadHtmlString` 的方法，可讓您將 HTML 字串做為引數。`WebView` 隨即會顯示提供的 HTML 網頁。在程式碼中新增下列方法：

lib/src/menu.dart

```
Future<void> _onLoadFlutterAssetExample(
  WebViewController controller, BuildContext context) async {
  await controller.loadFlutterAsset('assets/www/index.html');
}

Future<void> _onLoadLocalFileExample(
  WebViewController controller, BuildContext context) async {
  final String pathToIndex = await _prepareLocalFile();

  await controller.loadFile(pathToIndex);
}

static Future<String> _prepareLocalFile() async {
  final String tmpDir = (await getTemporaryDirectory()).path;
  final File indexFile = File('$tmpDir/www/index.html');

  await Directory('$tmpDir/www').create(recursive: true);
  await indexFile.writeAsString(kExamplePage);

  return indexFile.path;
}

// Add here ...
Future<void> _onLoadHtmlStringExample(
  WebViewController controller, BuildContext context) async {
  await controller.loadHtmlString(kExamplePage);
}

// ... to here.
```

新增菜單品項

素材資源已設定完畢並可供使用，且所有功能的方法都已建立，現在可以更新選單。在 `_MenuOptions` 列舉中新增下列項目：

lib/src/menu.dart

```
enum _MenuOptions {  
  navigationDelegate,  
  userAgent,  
  javascriptChannel,  
  listCookies,  
  clearCookies,  
  addCookie,  
  setCookie,  
  removeCookie,  
  // Add from here ...  
  loadFlutterAsset,  
  loadLocalFile,  
  loadHtmlString,  
  // ... to here.  
}
```

現在列舉已更新，您可以新增選單選項，並將這些選項連結至您剛才新增的輔助方法。將 `_MenuState` 類別更新為以下內容：

[lib/src/menu.dart](#)

```

class _MenuState extends State<Menu> {
  final cookieManager = WebViewCookieManager();

  @override
  Widget build(BuildContext context) {
    return PopupMenuButton<_MenuOptions>(
      onSelect: (value) async {
        switch (value) {
          case _MenuOptions.navigationDelegate:
            await widget.controller
              .loadRequest(Uri.parse('https://youtube.com'));
          case _MenuOptions.userAgent:
            final userAgent = await widget.controller
              .runJavaScriptReturningResult('navigator.userAgent');
            if (!context.mounted) return;
            ScaffoldMessenger.of(context).showSnackBar(SnackBar(
              content: Text('$userAgent'),
            ));
          case _MenuOptions.javascriptChannel:
            await widget.controller.runJavaScript(''
var req = new XMLHttpRequest();
req.open('GET', "https://api.ipify.org/?format=json");
req.onload = function() {
  if (req.status == 200) {
    let response = JSON.parse(req.responseText);
    SnackBar.postMessage("IP Address: " + response.ip);
  } else {
    SnackBar.postMessage("Error: " + req.status);
  }
}
req.send();''');
            case _MenuOptions.clearCookies:
              await _onClearCookies();
            case _MenuOptions.listCookies:
              await _onListCookies(widget.controller);
            case _MenuOptions.addCookie:
              await _onAddCookie(widget.controller);
            case _MenuOptions.setCookie:
              await _onSetCookie(widget.controller);
            case _MenuOptions.removeCookie:
              await _onRemoveCookie(widget.controller);
            case _MenuOptions.loadFlutterAsset: // Add from here
              if (!mounted) return;
              await _onLoadFlutterAssetExample(widget.controller, context);
            case _MenuOptions.loadLocalFile:
              if (!mounted) return;
              await _onLoadLocalFileExample(widget.controller, context);
            case _MenuOptions.loadHtmlString:
              if (!mounted) return;
              await _onLoadHtmlStringExample(widget.controller, context);
              // To here.
        }
      }
    );
  }
}

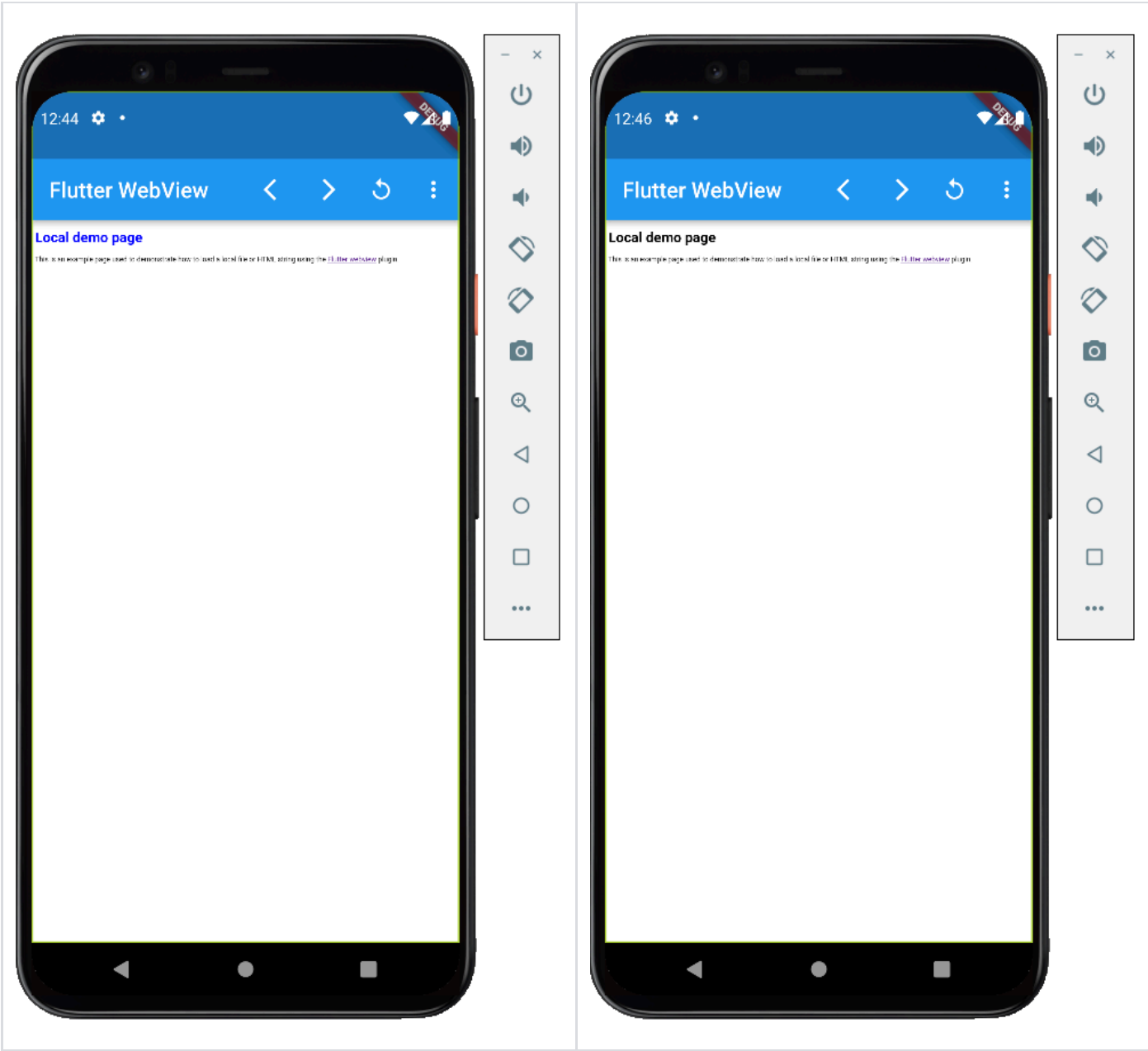
```

```

},
itemBuilder: (context) => [
  const PopupMenuItem<_MenuOptions>(
    value: _MenuOptions.navigationDelegate,
    child: Text('Navigate to YouTube'),
  ),
  const PopupMenuItem<_MenuOptions>(
    value: _MenuOptions.userAgent,
    child: Text('Show user-agent'),
  ),
  const PopupMenuItem<_MenuOptions>(
    value: _MenuOptions.javascriptChannel,
    child: Text('Lookup IP Address'),
  ),
  const PopupMenuItem<_MenuOptions>(
    value: _MenuOptions.clearCookies,
    child: Text('Clear cookies'),
  ),
  const PopupMenuItem<_MenuOptions>(
    value: _MenuOptions.listCookies,
    child: Text('List cookies'),
  ),
  const PopupMenuItem<_MenuOptions>(
    value: _MenuOptions.addCookie,
    child: Text('Add cookie'),
  ),
  const PopupMenuItem<_MenuOptions>(
    value: _MenuOptions.setCookie,
    child: Text('Set cookie'),
  ),
  const PopupMenuItem<_MenuOptions>(
    value: _MenuOptions.removeCookie,
    child: Text('Remove cookie'),
  ),
  const PopupMenuItem<_MenuOptions>(                                // Add from here
    value: _MenuOptions.loadFlutterAsset,
    child: Text('Load Flutter Asset'),
  ),
  const PopupMenuItem<_MenuOptions>(
    value: _MenuOptions.loadHtmlString,
    child: Text('Load HTML string'),
  ),
  const PopupMenuItem<_MenuOptions>(
    value: _MenuOptions.loadLocalFile,
    child: Text('Load local file'),
  ),
],
);
}

```


如要測試剛導入的程式碼是否正常運作，請在裝置上執行程式碼，然後按一下其中一個新加入的選單項目。請注意， `_onLoadFlutterAssetExample` 如何使用我們新增的 `style.css`，將 HTML 檔案的標題變更為藍色。



步驟 13 · 約 0 分鐘

13. 大功告成！

恭喜！！您已完成程式碼研究室。您可以在 [程式碼研究室存放區](#) 中找到本程式碼研究室的完整程式碼。

如要瞭解詳情，請嘗試其他 [Flutter 程式碼研究室](#)。
